

COMP 424 - Artificial Intelligence

Game Playing

Lecture 6: September 21, 2026

Jackie CK Cheung (jackie.cheung@mcgill.ca)

Su Lin Blodgett (sulin.blodgett@mcgill.ca)

Readings: R&N Ch 5

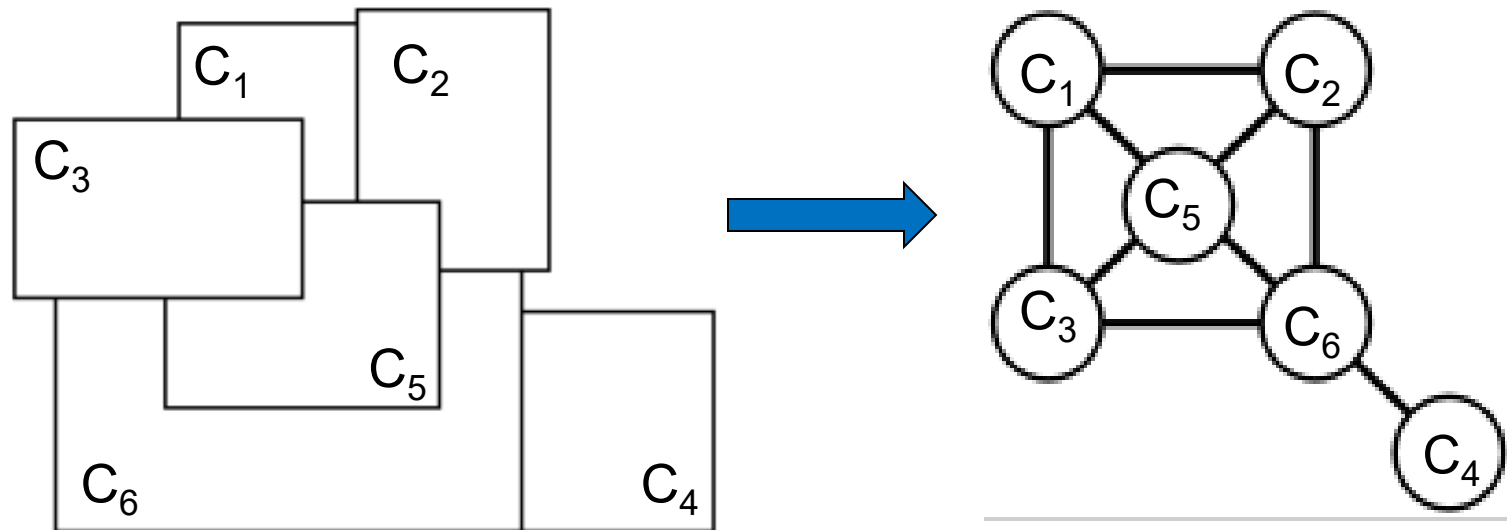
Announcements

- A1 is released on MyCourses!
 - Due on Sep 30, 2026 at **9pm**.

Review: Graph Colouring as a CSP

- Nodes are variables, arcs show constraints
- Graph structure can be exploited to accelerate solution search

E.g. Map colouring:

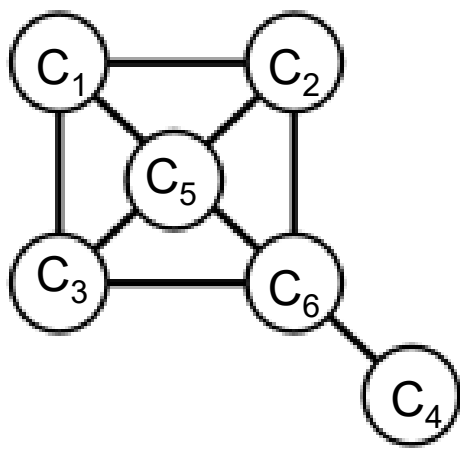


Review: Forward checking

Idea: Keep track of legal values for unassigned variables.

- When you assign a variable X
 - look at each unassigned variable Y connected to X (by a constraint)
 - delete from Y's domain any value that is inconsistent the value of X

E.g. Map coloring.



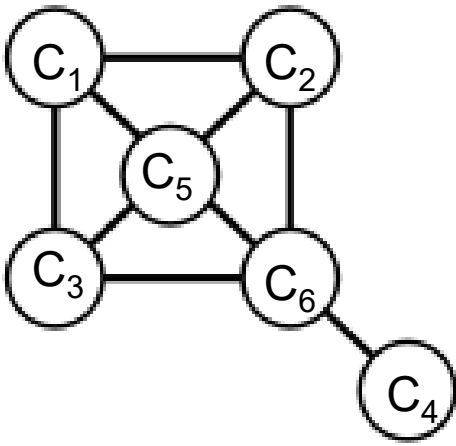
	Nothing assigned		
C1	RGB		
C2	RGB		
C3	RGB		
C4	RGB		
C5	RGB		
C6	RGB		

Forward checking

Idea: Keep track of legal values for unassigned variables.

- When you assign a variable X
 - look at each unassigned variable Y connected to X (by a constraint)
 - delete from Y's domain any value that is inconsistent the value of X

E.g. Map coloring.



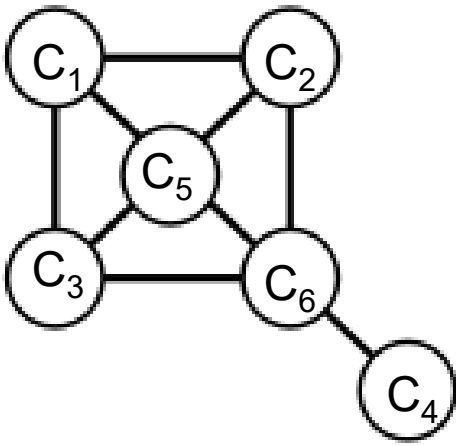
	Nothing assigned	Assign C1 = Red	
C1	RGB	R	
C2	RGB	GB	
C3	RGB	GB	
C4	RGB	RGB	
C5	RGB	GB	
C6	RGB	RGB	

Forward checking

Idea: Keep track of legal values for unassigned variables.

- When you assign a variable X
 - look at each unassigned variable Y connected to X (by a constraint)
 - delete from Y's domain any value that is inconsistent the value of X

E.g. Map coloring.



	Nothing assigned	Assign C1 = Red	Assign C2 = G
C1	RGB	R	R
C2	RGB	GB	G
C3	RGB	GB	GB
C4	RGB	RGB	RGB
C5	RGB	GB	B by forward checking!
C6	RGB	RGB	RB

Quick recap

Standard assumptions:

- **Discrete** (vs continuous) state space
- **Deterministic** (vs stochastic) environment
- **Observable** (vs unobservable) environment
- **Static** (vs changing) environment
- **There is only a single AI agent**

Quick recap

Standard assumptions:

- **Discrete** (vs continuous) state space
- **Deterministic** (vs stochastic) environment
- **Observable** (vs unobservable) environment
- **Static** (vs changing) environment
- **There is only a single AI agent**
 - Maybe we could simply model games as “uncertainty” in the environment? Chess opponents may be unpredictable.
 - Doesn't account for the fact that **the other player has its own goal**.
While somewhat unknown, we can compute over their decisions too!

Today's lecture

- Adversarial search → games with **2 players**
 - Planning ahead in a world where other agents are planning **against** us
- Minimax search
- Evaluation functions
- Alpha-beta pruning
- State-of-the-art game playing programs

Game playing

- One of the oldest, best studied domains in AI! Why?
 - They're fun! People enjoy games and are good at playing them
 - Many games are hard:
 - State spaces can be huge and complicated
 - Chess has branching factor of 35 and games go to 50 moves per player $\rightarrow 35^{100}$ nodes in the search tree!
 - Games may be **stochastic** and **partially observable**
 - Real-time constraints (e.g., fixed amount of time between moves)
 - *Games require the ability to make some decision even when calculating the optimal decision is infeasible*
 - Clear, clean description of the environment and actions
 - Easy performance indicator: **winning** is everything

Types of games

- Perfect information vs. Imperfect information
 - **Perfect**: players observe the exact state of the game
 - **Imperfect**: information is hidden from players
 - *Fully observable vs. partially observable*
- Deterministic vs. Stochastic
 - **Deterministic**: state changes are fully determined by player moves
 - **Stochastic**: state changes are partially determined by chance

What type of game?

Perfect information?

Yes

No

Deterministic

Checkers
Othello
Chess
Go

Mastermind
Solitaire*

Stochastic

Backgammon

Scrabble
Monopoly
Poker
Most card games

Game playing as search

- Consider 2-player, turn-taking, perfect information, deterministic games
- Can we formulate them as a search problem?

Game playing as search

- Consider 2-player, turn-taking, perfect information, deterministic games
- Can we formulate them as a search problem? **Yes!**

Game playing as search

- Consider 2-player, turn-taking, perfect information, deterministic games
- Can we formulate them as a search problem? Yes!

State, s : the state of the board + which player moves

Operators, o : Legal moves in a given state

Transition fcn: Defines the result of a move

Terminal states: States in which the game is over (won/lost/drawn)

Utility fcn: Defines a numeric value for the game for player p , when the game ends in terminal state s .

Simple case: +1 for win, -1 for loss, 0 for draw.

More complex cases: points won, money, ...

Game playing as search

- Consider 2-player, turn-taking, perfect information, deterministic games
- Can we formulate them as a search problem? Yes!
- We want to find a **strategy** (a way of picking moves) that maximizes **utility**
 - i.e., maximize the probability of winning or the expected points, minimize the cost, ...

Game search challenge

- It's not quite the same as simple searching
- There's an opponent! That opponent is **adversarial**!

Game search challenge

- It's not quite the same as simple searching
- There's an opponent! That opponent is **adversarial**!



- The opponent has its own goals which don't match our goals
- Opponent tries to make things good for itself and bad for us
- We must simulate the opponent's decisions

Zero-Sum Games

- A special case is where one player's gain equally balances the other player's loss: the **zero-sum** game
- In order to reason about this, introduce **utility**, a function over outcomes of the game, in Player 1's point-of-view.
 - Let high utility mean player 1 is happy (so player 2 is sad)
 - Let low utility mean player 2 is happy (so player 1 is sad)

Zero-Sum Games

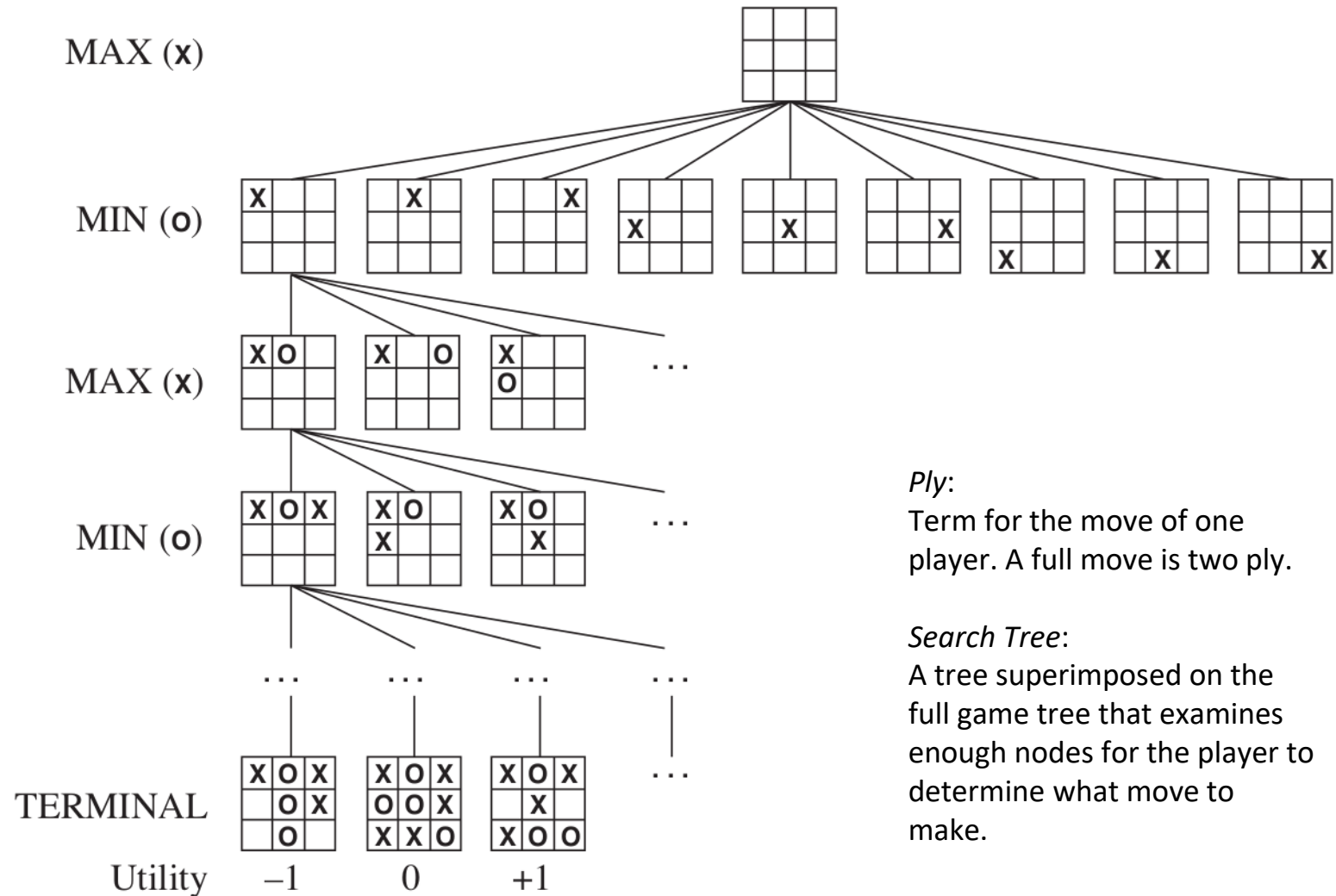


- Zero-sum games
 - Each player's gain or loss of utility is **exactly balanced** by the losses or gains of the utility of the other player
 - If I win, you lose: $U(p1) = 1 \rightarrow U(p2) = -1$
 $U(p1) + U(p2) = 0$
 - Important problem within Game Theory, where we can use some famous concepts when reasoning about our solutions:
 - Minimax Theorem of von Neuman 1928
 - Nash Equilibria 1951

Game search challenge

- It's not quite the same as simple searching
- There's an opponent! That opponent is **adversarial**!
 - It has its own goal which does not match our goal
 - We must simulate the opponent's decisions
- **Key idea:** Define
 - a **max player** (who wants to maximize the utility)
 - a **min player** (who wants to minimize it)

Example: Game Tree for Tic-Tac-Toe



Minimax search

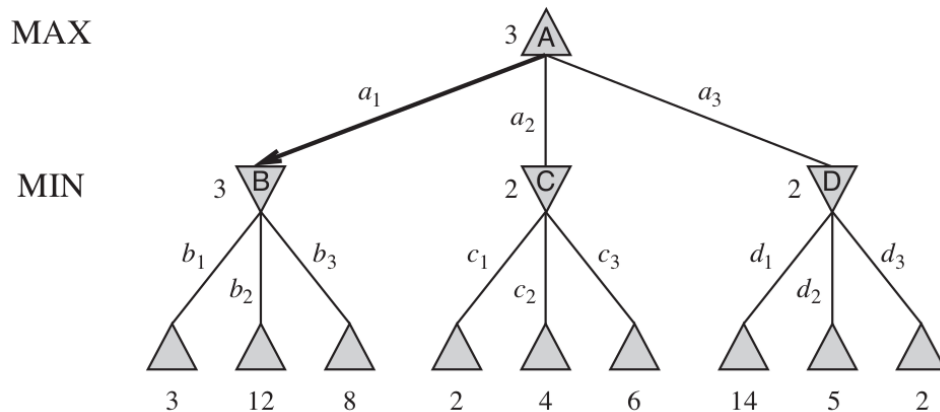
An algorithm for finding the optimal strategy: the best move to play at each state (node)

How it works:

- Expand the complete search tree until terminal states have been reached
- Compute utilities of the terminal states
- **Back up** from the leaves towards the current game state:
 - At each min node: back up the worst value among its children
 - At each max node: back up the best value among its children

Minimax search

- Expand the search tree to the terminal states
- Compute utilities at terminal states
- Back up from the leaves towards the current game state
 - At each min node: back up the worst value among the children
 - At each max node: back up the best value among the children



MINIMAX(Node n) =

$$\begin{cases} \text{UTILITY}(n) & \text{if } n \text{ is a terminal state} \\ \max_{s \in \text{Succ}(n)} \text{MINIMAX}(s) & \text{if } n \text{ is a MAX node} \\ \min_{s \in \text{Succ}(n)} \text{MINIMAX}(s) & \text{if } n \text{ is a MIN node} \end{cases}$$

Minimax search

MINIMAX(Node n) =

$$\begin{cases} \text{UTILITY}(n) & \text{if } n \text{ is a terminal state} \\ \max_{s \in \text{Succ}(n)} \text{MINIMAX}(s) & \text{if } n \text{ is a MAX node} \\ \min_{s \in \text{Succ}(n)} \text{MINIMAX}(s) & \text{if } n \text{ is a MIN node} \end{cases}$$

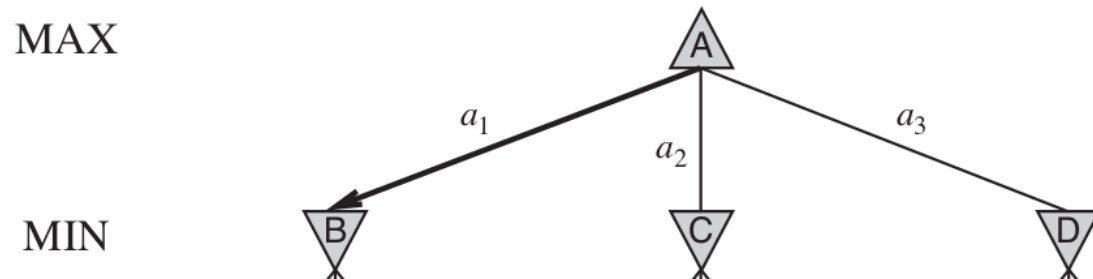
MAX



Minimax search

MINIMAX(Node n) =

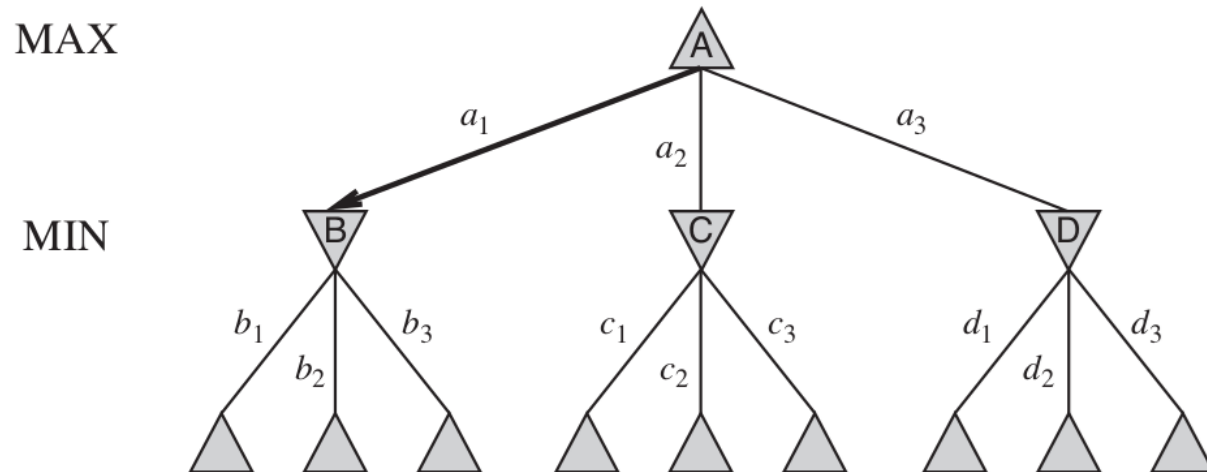
$$\begin{cases} \text{UTILITY}(n) & \text{if } n \text{ is a terminal state} \\ \max_{s \in \text{Succ}(n)} \text{MINIMAX}(s) & \text{if } n \text{ is a MAX node} \\ \min_{s \in \text{Succ}(n)} \text{MINIMAX}(s) & \text{if } n \text{ is a MIN node} \end{cases}$$



Minimax search

MINIMAX(Node n) =

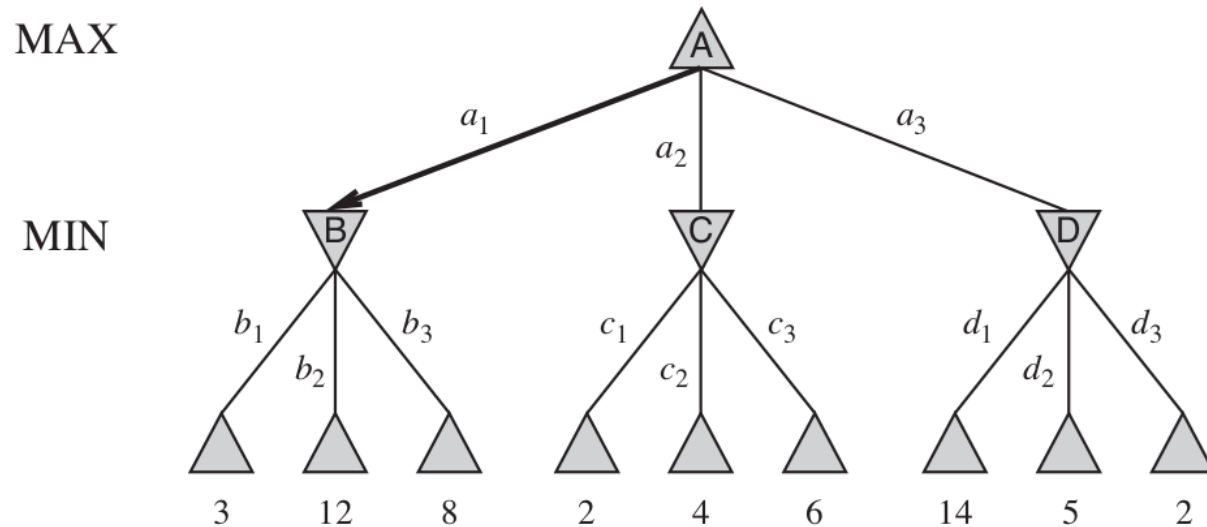
$$\begin{cases} \text{UTILITY}(n) & \text{if } n \text{ is a terminal state} \\ \max_{s \in \text{Succ}(n)} \text{MINIMAX}(s) & \text{if } n \text{ is a MAX node} \\ \min_{s \in \text{Succ}(n)} \text{MINIMAX}(s) & \text{if } n \text{ is a MIN node} \end{cases}$$



Minimax search

MINIMAX(Node n) =

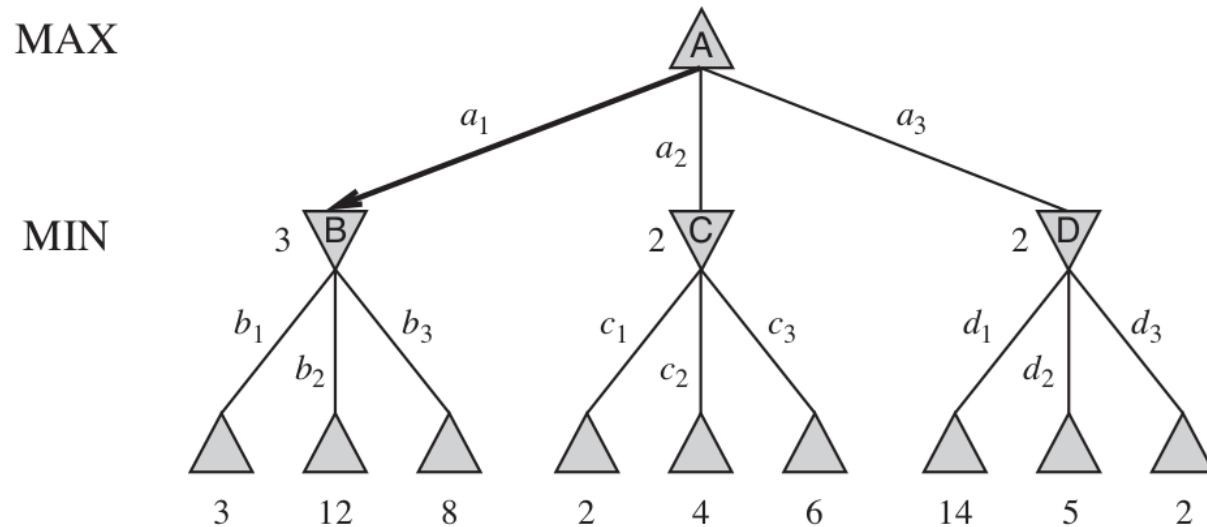
$$\begin{cases} \text{UTILITY}(n) & \text{if } n \text{ is a terminal state} \\ \max_{s \in \text{Succ}(n)} \text{MINIMAX}(s) & \text{if } n \text{ is a MAX node} \\ \min_{s \in \text{Succ}(n)} \text{MINIMAX}(s) & \text{if } n \text{ is a MIN node} \end{cases}$$



Minimax search

MINIMAX(Node n) =

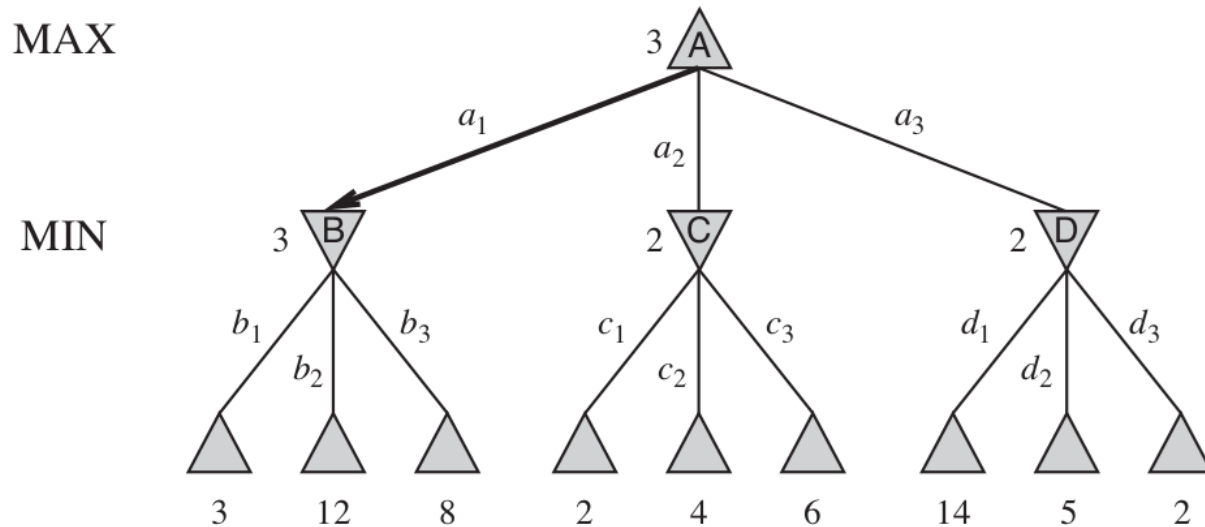
$$\begin{cases} \text{UTILITY}(n) & \text{if } n \text{ is a terminal state} \\ \max_{s \in \text{Succ}(n)} \text{MINIMAX}(s) & \text{if } n \text{ is a MAX node} \\ \min_{s \in \text{Succ}(n)} \text{MINIMAX}(s) & \text{if } n \text{ is a MIN node} \end{cases}$$



Minimax search

MINIMAX(Node n) =

$$\begin{cases} \text{UTILITY}(n) & \text{if } n \text{ is a terminal state} \\ \max_{s \in \text{Succ}(n)} \text{MINIMAX}(s) & \text{if } n \text{ is a MAX node} \\ \min_{s \in \text{Succ}(n)} \text{MINIMAX}(s) & \text{if } n \text{ is a MIN node} \end{cases}$$



The minimax value at each node tells us how to play an optimal game!

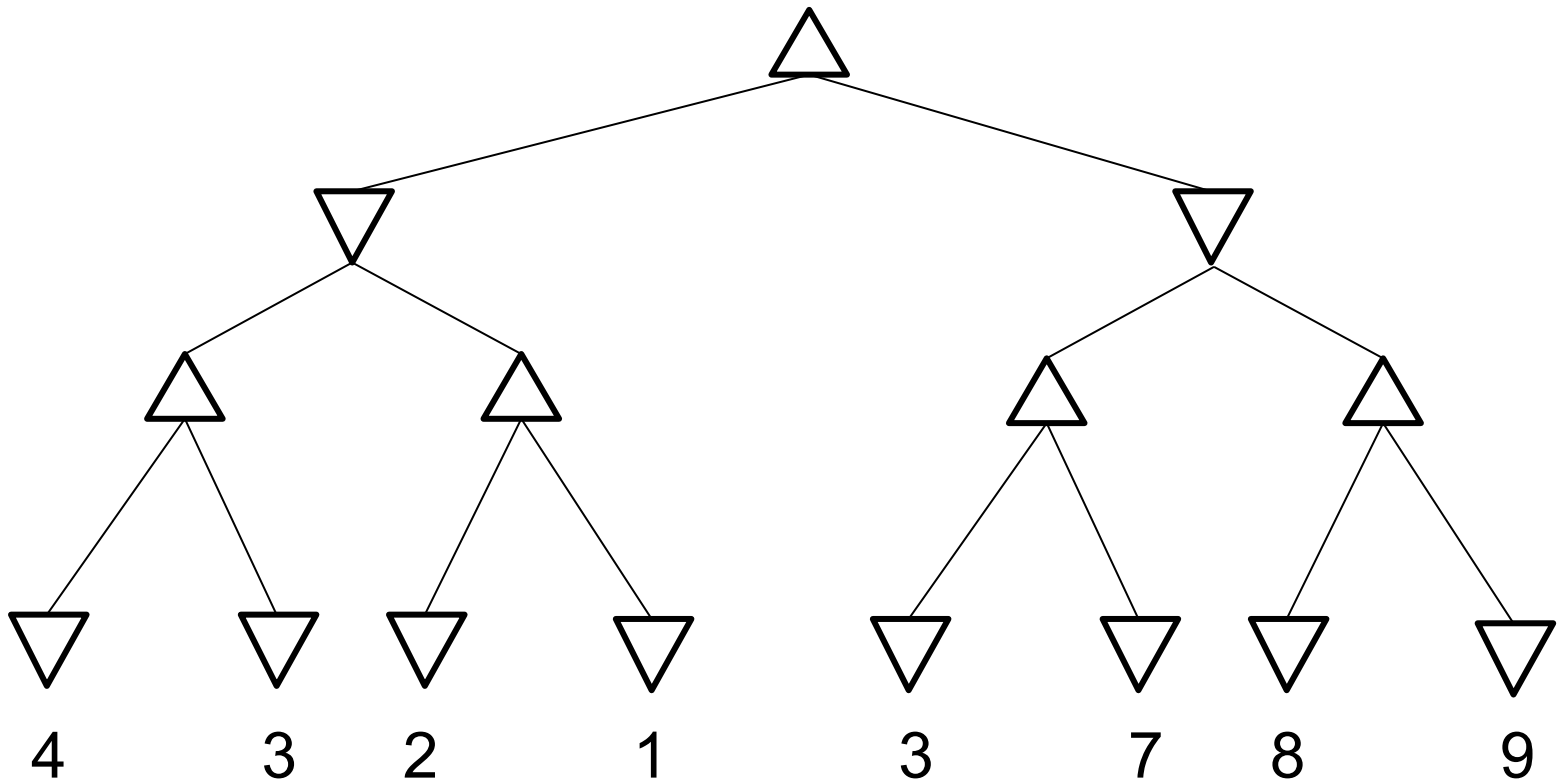
Minimax Algorithm

```
operator MinimaxDecision()
  for each legal operator o:
    apply the operator o and obtain the new
    game state s.
    Value[o] = MinimaxValue(s)
  return the operator o with the highest value
  Value[o].

double MinimaxValue(s)
  if isTerminal(s), return Utility(s).
  for each state s' in Successors(s)
    let Value(s') = MinimaxValue(s').
  if Max's turn to move in s, return  $\max_s, Value(s')$ .
  if Min's turn to move in s, return  $\min_s, Value(s')$ .
```

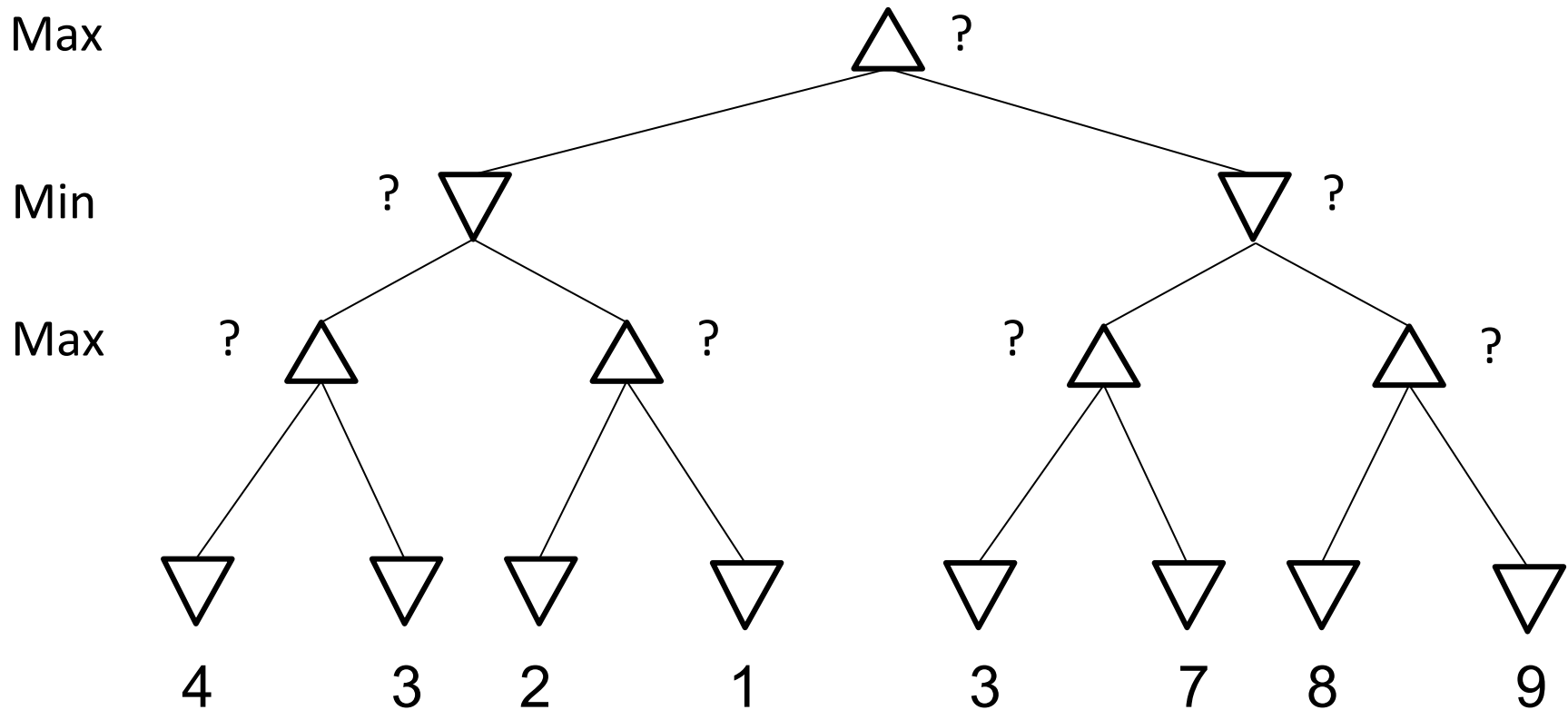
Exercise

- Apply minimax search to the following game search tree



Exercise

- Apply minimax search to the following game search tree



Properties of Minimax search

- Complete?
- Optimal?
- Time complexity?
- Space complexity?

Properties of Minimax search

- Complete? If the game tree is finite
- Optimal? Against an optimal opponent, yes
 - It maximizes the **worst-case** outcome for Max
 - There might exist better strategies for suboptimal opponents
- Time complexity? $O(b^m)$
- Space complexity? $O(bm)$ if we use DFS

Properties of Minimax search

- Complete? If the game tree is finite
- Optimal? Against an optimal opponent, yes
 - It maximizes the **worst-case** outcome for Max
 - There might exist better strategies for suboptimal opponents
- Time complexity? $O(b^m)$
- Space complexity? $O(bm)$ if we use DFS
- So is Minimax suitable for solving chess?

Properties of Minimax search

- Complete? If the game tree is finite
- Optimal? Against an optimal opponent, yes
 - It maximizes the **worst-case** outcome for Max
 - There could be superior strategies for suboptimal opponents
- Time complexity? $O(b^m)$
- Space complexity? $O(bm)$ if we use DFS
- So is Minimax suitable for solving chess?
 - In chess: $b \approx 35$, $m \approx 100$, so an exact solution is impossible!

Coping with resource limitations

- Suppose we have 100 seconds to make a move, and we can search 10^4 nodes per second
 - Then we can only search 10^6 nodes
(Or even fewer, if we spend time deciding which nodes to search)
- Possible approach:
 - Use a cutoff test (e.g., based on a depth limit)
 - Use an **evaluation function** for the nodes where we cut the search (since they're not terminal states, we don't know the true utility)
- Need to think about **real-time** search

Evaluation functions

- An evaluation function $v(s)$ **estimates** the expected utility of a state (e.g., the likelihood of winning from that state)
 - Performance depends strongly on the quality of $v(s)$
 - If it is too inaccurate it will lead the agent astray
- Desiderata:
 - It should order the terminal states according to their true utility
 - It should be relatively quick to compute
 - For nonterminal states, it should be strongly correlated with the actual chances of winning
- An evaluation function can be designed by an expert or **learned** from experience

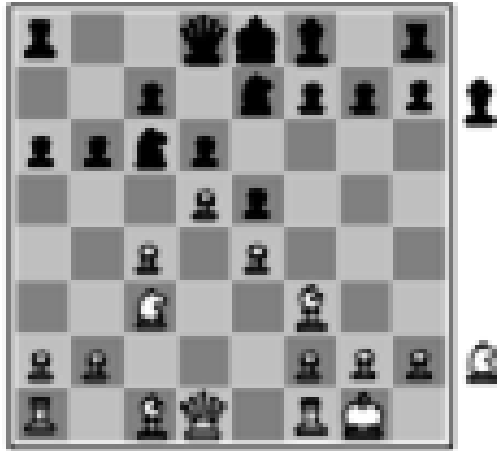
Evaluation functions

- Most evaluation functions work by calculating **features** of the state (e.g., # of pawns, queens, etc.)
- Features define **categories** or **equivalence classes** of states
- Typically, we compute features separately and then combine them for an aggregate value
- E.g., if the features of the board are **independent**, use a weighted linear function:

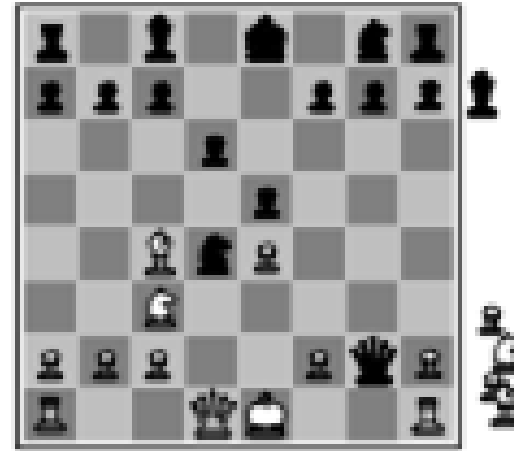
$$v(s) = w_1 f_1(s) + w_2 f_2(s) + \dots + w_n f_n(s)$$

- Independence is a strong, likely incorrect assumption. But it could still be useful!
 - An extra bishop is worth more in the endgame; its weight should depend on the “move number” feature

Example: Chess



Black to move
White slightly better



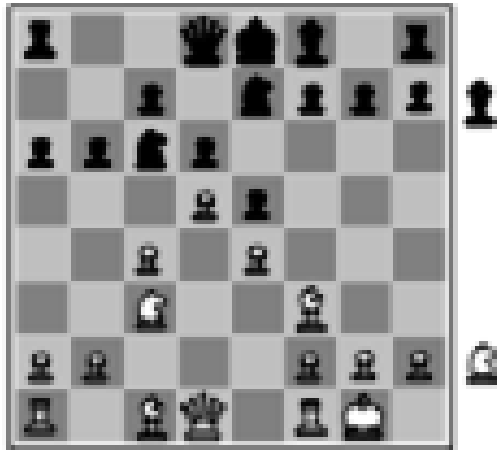
White to move
Black winning

- Linear evaluation function: $v(s) = w_1 f_1(s) + w_2 f_2(s)$

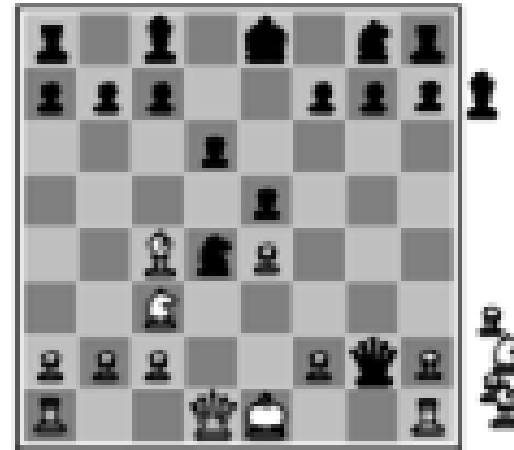
$$f_1(s) = (\# \text{ white queens}) - (\# \text{ black queens})$$

$$f_2(s) = (\# \text{ white pawns}) - (\# \text{ black pawns})$$

Example: Chess



Black to move
White slightly better

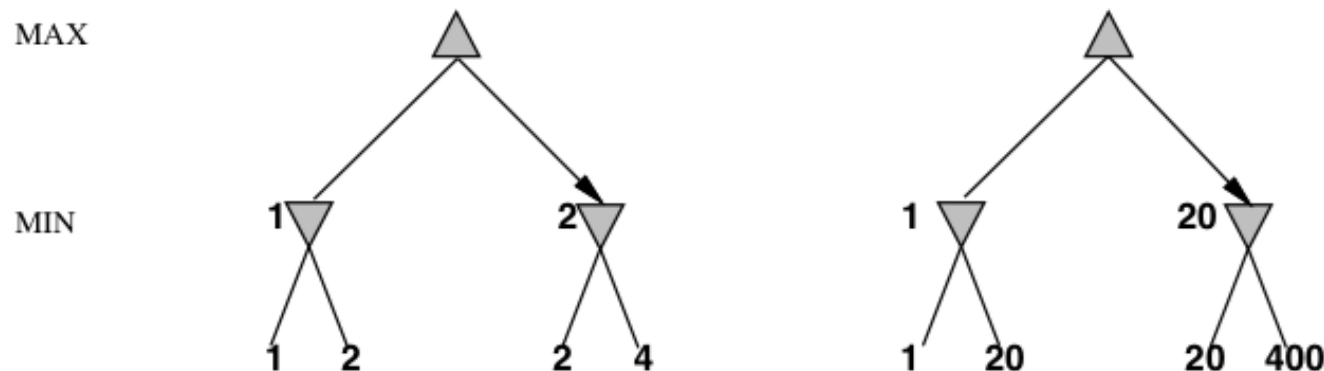


White to move
Black winning

- Linear evaluation function: $v(s) = w_1 f_1(s) + w_2 f_2(s)$
 $f_1(s) = (\text{\# white queens}) - (\text{\# black queens})$ $w_1 = 9$
 $f_2(s) = (\text{\# white pawns}) - (\text{\# black pawns})$ $w_2 = 3$

How precise should the evaluation fcn be?

- For deterministic games, all that matters is that the fcn preserve the ordering of the nodes
- Thus, the move chosen is invariant under **monotonic transformations** of the evaluation function



- In deterministic games, payoff acts as an ordinal utility function

Minimax with an evaluation function

- Use the **evaluation function** to evaluate non-terminal nodes
 - This helps make a decision without searching until the end of the game
- **Minimax cutoff algorithm:**

Same as standard Minimax, except stop at some maximum depth m , use the evaluation function on those nodes, back up from there

Minimax cutoff in chess

- How many moves ahead can we search in chess?
If our hardware could search 10^6 nodes in the available time, then minimax cutoff with $b=35$ could search **4** moves ahead
- Is that good?
 - 4 moves ahead $\approx$ novice player
 - 8 moves ahead $\approx$ human master, typical PC
 - 12 moves ahead $\approx$ Deep Blue, Kasparov
- Key idea:
Instead of exhaustive search, let's search a few lines of play, but deeply
We need pruning!

Conclusion

- Minimax search is a powerful algorithm, and all you need for small games.
 - Captures adversarial, two-player, zero-sum nature
 - Does all the “thinking” for both you and an optimal opponent
 - Important note: Not the best way to exploit your friends’ weak play (WHY?)
- Minimax still does not scale to large problems. We will see better methods coming up!